TASK 10 – Compare Expectations with Execution

Expected behaviour

For 300, 6,000 and 150,000 payments, Decimal totals should preserve the exact decimal values represented by the input strings. Float totals can differ by small fractions of a paisa due to binary representation and accumulation order. The exact observed difference and elapsed time depend on the machine, Python version, values and test data; they must be measured on the student's computer rather than invented in the report.

Measurement program

```
from decimal import Decimal
from time import perf_counter

PAYMENT_TEXT = "33.40"

def measure(count):
    values = [PAYMENT_TEXT] * count
    start = perf_counter()
    float_total = sum(float(x) for x in values)
    float_seconds = perf_counter() - start
    start = perf_counter()
    decimal_total = sum((Decimal(x) for x in values), Decimal("0.00"))
    decimal_seconds = perf_counter() - start
    difference = Decimal(str(float_total)) - decimal_total
    print(f"n={count:>6} float={float_total:.12f} Decimal={decimal_total} diff={difference} "
          f"float_s={float_seconds:.6f} decimal_s={decimal_seconds:.6f}")

for size in (300, 6000, 150000):
    measure(size)
```

Rounding policy

Rounding every payment to two decimal places before adding it gives a sum of payable paisa amounts and is appropriate when each farmer's individual transaction is settled to the nearest paisa. Rounding only the final total can differ when individual products contain fractions of a paisa. The co-operative must define whether it settles each delivery, each farmer's ten-day total, or another unit. The implementation should follow that policy consistently. The recommended approach is Decimal from text, an explicit rounding mode, and a documented settlement point.

Rate-table alternative

If the same (fat, SNF) pairs recur, compute their rate once and cache it in a dictionary keyed by the pair. Average dictionary lookup is O(1); computing the rate also consists of a fixed number of arithmetic operations and is O(1), so the asymptotic complexity does not improve for this simple formula. Caching can reduce repeated Decimal operations when there are many repeated pairs, at the cost of O(k) extra memory for k distinct pairs. Use time.perf_counter() or timeit to measure whether the optimisation matters for the actual workload.

Measurement evidence note

Run the supplied program and paste its output into the submitted report. Do not claim fabricated timings. Repeat measurements several times and compare medians if more reliable timing is required.